

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101
--------------	---	-----------------	---------

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I Yoga Madha A-192425058(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

1. Medical Diagnosis Expert System Using Prolog

Problem Statement

Develop a rule-based expert system that identifies possible diseases based on patient symptoms and basic clinical observations. The system represents medical knowledge using Prolog facts and production rules and performs reasoning using forward chaining and backward chaining. The system is intended for **academic demonstration of expert-system concepts**, not for real medical diagnosis.

Objectives

- Represent patient information using Prolog facts.
- Develop a medical knowledge base using production rules.
- Implement forward chaining.
- Demonstrate backward chaining.
- Demonstrate unification and backtracking.
- Generate possible disease conclusions automatically.
- Test the system using different patient cases.
- Compare procedural and non-procedural approaches.

Domain Analysis

Domain

Medical Diagnosis

Input

The system accepts information such as:

- Fever
- Cough
- Sore throat
- Body pain
- Headache
- Rash
- Joint pain
- Sneezing
- Runny nose
- Breathing difficulty
- Chest pain
- Nausea
- Vomiting
- Diarrhea
- Stomach pain
- Mucus
- Fatigue

Output

The system identifies possible diseases such as:

- Flu
- Common Cold
- Dengue
- Pneumonia
- Migraine
- Food Poisoning
- Bronchitis

Knowledge Representation

Knowledge is represented using **facts and production rules**.

Facts

symptom(patient1, fever).

symptom(patient1, cough).

symptom(patient1, sore_throat).

symptom(patient1, body_pain).

These facts indicate that patient1 has fever, cough, sore throat and body pain.

Production Rule

disease_for_patient(Patient, flu) :-

 symptom(Patient, fever),

 symptom(Patient, cough),

 symptom(Patient, sore_throat).

Meaning:

IF patient has fever

AND patient has cough

AND patient has sore throat

THEN patient may have flu.

Production Rules

The project contains rules for different diseases.

Flu

disease_for_patient(Patient, flu) :-

 symptom(Patient, fever),

 symptom(Patient, cough),

 symptom(Patient, sore_throat).

disease_for_patient(Patient, flu) :-

 symptom(Patient, fever),

 symptom(Patient, body_pain),

 symptom(Patient, headache).

Common Cold

disease_for_patient(Patient, common_cold) :-

 symptom(Patient, cough),

 symptom(Patient, sneezing),

 symptom(Patient, runny_nose).

disease_for_patient(Patient, common_cold) :-
 symptom(Patient, runny_nose),
 symptom(Patient, sneezing),
 symptom(Patient, sore_throat).

Dengue

disease_for_patient(Patient, dengue) :-
 symptom(Patient, fever),
 symptom(Patient, headache),
 symptom(Patient, rash),
 symptom(Patient, joint_pain).

disease_for_patient(Patient, dengue) :-
 symptom(Patient, fever),
 symptom(Patient, body_pain),
 symptom(Patient, rash).

Pneumonia

disease_for_patient(Patient, pneumonia) :-
 symptom(Patient, fever),
 symptom(Patient, cough),
 symptom(Patient, breathing_difficulty).

disease_for_patient(Patient, pneumonia) :-
 symptom(Patient, cough),
 symptom(Patient, chest_pain),
 symptom(Patient, breathing_difficulty).

Migraine

disease_for_patient(Patient, migraine) :-
 symptom(Patient, headache),
 symptom(Patient, nausea),
 symptom(Patient, light_sensitivity).

disease_for_patient(Patient, migraine) :-
 symptom(Patient, severe_headache),
 symptom(Patient, nausea),
 symptom(Patient, light_sensitivity).

Food Poisoning

disease_for_patient(Patient, food_poisoning) :-
 symptom(Patient, vomiting),
 symptom(Patient, diarrhea),
 symptom(Patient, stomach_pain).

disease_for_patient(Patient, food_poisoning) :-
 symptom(Patient, nausea),

```
symptom(Patient, vomiting),
symptom(Patient, diarrhea).
```

Bronchitis

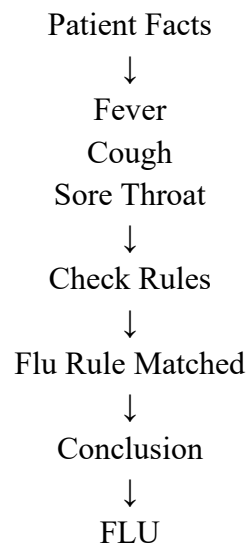
```
disease_for_patient(Patient, bronchitis) :-
    symptom(Patient, cough),
    symptom(Patient, mucus),
    symptom(Patient, chest_discomfort).
```

```
disease_for_patient(Patient, bronchitis) :-
    symptom(Patient, persistent_cough),
    symptom(Patient, mucus),
    symptom(Patient, fatigue).
```

Forward Chaining

Forward chaining is **data-driven reasoning**.

The system starts with the available patient facts and checks which production rules can be satisfied.



Prolog command

```
?- show_forward(patient1).
```

Expected output

FORWARD CHAINING FOR patient1

Initial facts:

```
[fever,cough,sore_throat,body_pain]
```

Checking production rules...

Rule fired -> flu

Final conclusions -> [flu]

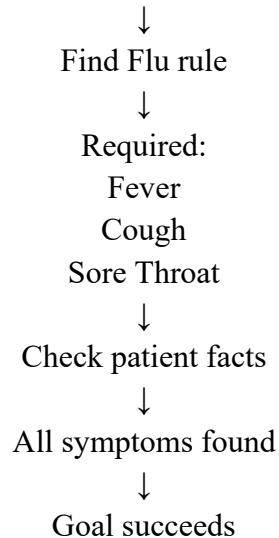
Backward Chaining

Backward chaining is **goal-driven reasoning**.

The system starts with a possible diagnosis and works backward to determine whether the required symptoms are present.

Example:

Goal: Is patient1 suffering from Flu?



Prolog command

?- show_backward(patient1, flu).

Expected output

BACKWARD CHAINING

Goal: Is patient1 suffering from flu?

Goal succeeded.

Evidence: required symptoms are present.

For an incorrect diagnosis:

?- show_backward(patient1, dengue).

Output:

BACKWARD CHAINING

Goal: Is patient1 suffering from dengue?

Goal failed.

Evidence: required symptoms are not all present.

Unification

Unification is the process by which Prolog matches terms and assigns values to variables.

Example:

symptom(Patient, fever).

Query:

?- symptom(patient1, fever).

Prolog matches:

Patient = patient1

Therefore, the variable Patient is unified with patient1.

Backtracking

Backtracking allows Prolog to search for alternative solutions when more than one rule or fact can satisfy a goal.

Example:

?- diagnosis(patient1, Disease).

Prolog searches through the available disease rules.

If one rule succeeds, Prolog can backtrack and check other rules for additional possible diagnoses.

This demonstrates automated logical search.

Test Cases

Test	Patient	Symptoms	Expected Result
1	patient1	Fever, cough, sore throat	Flu
2	patient2	Fever, headache, rash, joint pain	Dengue
3	patient3	Cough, sneezing, runny nose	Common Cold
4	patient1	Goal = Flu	Success
5	patient1	Goal = Dengue	Failure

Sample Test Commands

Load the files:

?- [medical_expert].

?- [forward_chaining].

?- [backward_chaining].

?- [test_cases].

Run forward chaining:

?- show_forward(patient1).

Run backward chaining:

?- show_backward(patient1, flu).

Test an incorrect diagnosis:

?- show_backward(patient1, dengue).

Run all test cases:

?- run_tests.

Procedural vs Non-Procedural

Procedural Programming	Non-Procedural / Declarative Programming
Describes how to solve a problem	Describes facts and relationships

Procedural Programming	Non-Procedural / Declarative Programming
Uses step-by-step instructions	Uses facts and rules
Programmer controls execution	Inference engine performs reasoning
Focus is on procedure	Focus is on knowledge
Example: Python, C	Example: Prolog

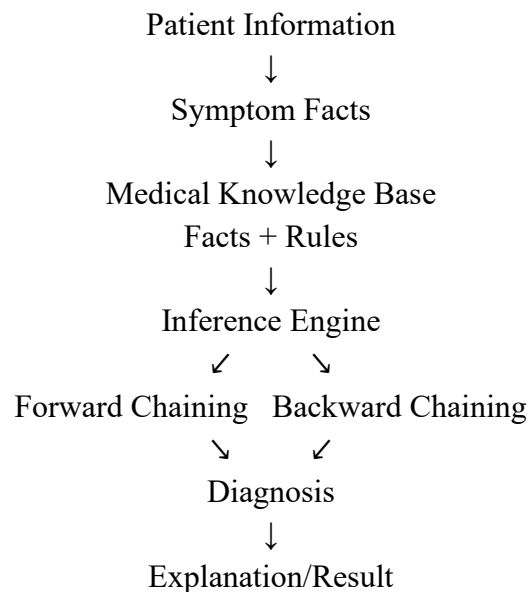
Why Prolog?

Prolog is suitable because it allows knowledge to be represented naturally using:

Facts + Rules + Queries

The Prolog inference engine automatically performs logical reasoning using **unification and backtracking**.

System Architecture



Contents of Each .pl File

medical_expert.pl

Contains:

Patient facts

Symptom facts

Disease rules

Diagnosis predicate

Explanation facility

Interactive system

forward_chaining.pl

Contains:

Production rule representation

Rule matching

Condition checking

Forward chaining engine

Forward reasoning demonstration

backward_chaining.pl

Contains:

Goal-driven diagnosis

Backward reasoning

Successful goal demonstration

Failed goal demonstration

test_cases.pl

Contains:

Patient test cases

Forward chaining tests

Backward chaining tests

Successful diagnosis tests

Failed diagnosis tests

Evaluation

Correctness

The system correctly matches predefined symptoms with the corresponding rules.

Knowledge Coverage

The system contains multiple disease categories and multiple symptom combinations.

Reasoning Accuracy

The system uses logical inference to derive conclusions from available facts.

Forward Chaining

Starts from patient symptoms and moves toward conclusions.

Backward Chaining

Starts from a diagnosis goal and searches for supporting symptoms.

Prolog Features

The system demonstrates:

- Facts
- Rules
- Predicates
- Variables
- Unification
- Backtracking
- Logical inference

Limitations

- The medical rules are simplified.
- The system uses a limited number of diseases.
- Laboratory results are not included.
- Patient medical history is limited.
- The system does not replace a qualified medical professional.
- It is intended only for academic demonstration.

Future Enhancements

- Add more diseases.
- Add more symptoms.
- Add patient medical history.
- Add severity classification.
- Add explanation traces for every rule.
- Add a graphical user interface.
- Add a larger validated medical knowledge base.
- Add multilingual interaction.

Conclusion

The Medical Diagnosis Expert System demonstrates the development of a rule-based expert system using SWI-Prolog. Medical knowledge is represented through facts and production rules. The system performs forward chaining from known patient symptoms and backward chaining from diagnosis goals. Prolog's unification and backtracking mechanisms support automated logical inference and alternative solution searching.

The project therefore demonstrates the major concepts required for constructing and evaluating a rule-based expert system.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.

3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.

7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				